Keystone Connect Production Technical Blueprint

1) Objective

Build Keystone Connect into a production-grade, secure, observable, and scalable contractor discovery platform with stable UX across desktop/mobile, reliable Google Places integration, and controlled data quality workflows.

2) Current Baseline (As-Built)

Codebase is a Node.js monolith serving API + UI from one process.

- Runtime: Node `24.x` (`package.json`, `render.yaml`)
- Server: `/Users/mcdowell/Desktop/temp files/Keystone Connect/src/web-server.js`
- Search engine: `/Users/mcdowell/Desktop/temp files/Keystone Connect/src/search-engine.js`
- Taxonomy: `/Users/mcdowell/Desktop/temp files/Keystone Connect/data/taxonomy.json`
- Primary UI: `/Users/mcdowell/Desktop/temp files/Keystone Connect/ui-v2/app.html`
- Persistence (JSON files):
- `/Users/mcdowell/Desktop/temp files/Keystone Connect/data/irrelevant-filters.json`
- `/Users/mcdowell/Desktop/temp files/Keystone Connect/data/preferred-results.json`

Current routes:

- App routes: `/`, `/v2`, `/ui-v2`, `/ui-v2/app.html`, fallback `/v1`
- Search/data APIs:
- `POST /api/search`
- `GET /api/query-categories`
- `GET /api/location-suggest`
- `GET /api/location-city`
- `GET /api/reverse-zip`
- `GET /api/preferred`
- `POST /api/preferred`
- `POST /api/irrelevant`
- `POST /api/csv` (legacy CSV endpoint; UI exports `.xlsx` client-side)
- Health: `GET /api/ping`

3) Core Functional Flows

3.1 Search

1. User enters location or geolocation coordinates.
2. User selects category/child/manual query.
3. UI calls `POST /api/search`.

4 Server resolves center (lat/lng direct or Places geocode by text).
5 Engine builds jobs:
6 `API` mode: taxonomy-driven job generation (`buildQueries`), weighted by parent/child query profiles.
7 `GGL` mode: broader legacy query expansion (`buildLegacyQueries`).
8 Engine executes Google Places jobs with retries and concurrency.
9 Results are deduped, filtered, ranked, radius-clamped, and returned.

3.2 Quality controls

- Prior ON/OFF: client-side filtering of previously seen results in current expansion chain.
- Irrelevant removal: search-signature scoped suppression stored server-side in JSON.
- Preferred results: persistent star list stored server-side in JSON.

3.3 Export

- UI exports `.xlsx` in-browser using generated OOXML ZIP.
- Export file name uses selected category + location slug.

4) Search/Ranking Logic (Current)

From `/src/search-engine.js`:

- Google Places API (New) endpoints:
- `places:searchText`
- `places:searchNearby` (when taxonomy profile mode is `NEARBY_TYPES`)
- Field mask is explicitly set and includes IDs, address, types, phone, site, maps URL, rating.
- Concurrency: `MAX_JOB_CONCURRENCY = 4`
- Retry: exponential backoff on `429` and `5xx`, up to 4 attempts.
- Radius enforcement:
- Search radius capped at 45 mi in app.
- Google request radius capped at 50,000m.
- Hard excludes: taxonomy defaults and text blob filtering.
- Soft excludes: global + profile terms with score penalties.
- Score factors:
- Type hint match
- Name/category hint match
- Website presence
- Rating count threshold
- Rating threshold
- Proximity bonus

- Child priority weight multiplier

5) Production Target Architecture

5.1 Service layout

Phase target remains simple but hardened:

1. `web` service (Node API + static UI) behind TLS
2. Managed datastore for stateful records (replace JSON files)
3. Optional worker queue for email enrichment
4. Observability stack (logs, metrics, error tracking)

5.2 Data layer migration

Replace file-based JSON with relational storage.

Suggested tables:

- `projects` (future multi-project support)
- `search_runs` (signature, inputs, engine mode, timings)
- `search_results` (place snapshot per run)
- `suppressed_results` (scope: project/signature, key, actor, timestamp)
- `preferred_results` (scope: project/global, key, metadata, actor)
- `audit_events` (who changed what)

Recommended DB:

- PostgreSQL (Render managed Postgres or equivalent)

5.3 Caching

- Add short-lived cache for category payload and location suggestions.
- Add keyed cache for identical search requests to reduce API spend.
- Cache key should include:
- Engine mode
- Location center
- Radius
- Query or selected taxonomy node set
- Include-email flag

6) Security and Compliance

6.1 Secrets

- Keep `GOOGLE_MAPS_API_KEY` in environment only.

- Restrict key to required Google APIs and host/IP rules.
- Rotate API key on schedule and after any exposure.

6.2 API protections

- Add rate limiting per IP/session for `POST /api/search`.
- Add payload validation schema for all mutating endpoints.
- Add CORS allowlist if non-same-origin clients are introduced.

6.3 PII and data retention

- Contact data is business/public data but still needs retention policy.
- Define retention windows for:
- search logs
- suppression actions
- preferred list changes

7) Reliability and Scalability

7.1 SLO targets

- API availability: `99.9%`
- P95 search API latency:
- `<= 3.5s` without email enrichment
- `<= 12s` with email enrichment
- Error budget:
- `< 1%` failed search requests (excluding invalid user input)

7.2 Failure handling

- Continue partial result return when some Google jobs fail.
- Surface clear user-facing status:
- `partial results`
- `provider timeout`
- `retry suggested`
- Add circuit-breaker style throttle when upstream returns sustained `429`.

7.3 Horizontal scale

- Node service stateless except current JSON files.
- After DB migration, scale web replicas horizontally.
- Move enrichment work to async worker queue.

8) API Contract Hardening

Add explicit OpenAPI spec and enforce request/response contracts.

Minimum schema validation:

- `/api/search`:
- required location
- radius bounds
- engine mode enum
- query arrays length limits
- `/api/preferred`, `/api/irrelevant`:
- required key/action fields
- action enum
- max string lengths

9) UX/Frontend Architecture

9.1 Current

- Single-file app (`ui-v2/app.html`) with embedded CSS/JS.
- Session persistence via `sessionStorage`.
- Responsive layout with dedicated mobile media rules.

9.2 Production refinements

- Split frontend into modular JS files:
- state manager
- API client
- rendering layer
- interaction handlers
- Add shared design tokens file for consistent theming.
- Add strict accessibility pass:
- keyboard flow
- focus states
- contrast checks
- touch target sizing

10) Quality Engineering Plan

10.1 Automated tests

- Unit tests:

- taxonomy load and selection
- query builder
- scoring and filtering
- signature/key builders
- Integration tests:
- `/api/search` happy path + error paths
- `/api/preferred` add/remove
- `/api/irrelevant` suppression behavior
- snapshot tests for `.xlsx` builder outputs

10.2 Regression pack

- Gold test set by city/category:
- expected minimum result counts
- expected known-true businesses present
- known-false businesses excluded

10.3 Performance tests

- Load test search endpoint at realistic concurrency.
- Track Google API call volume per user action.

11) DevOps and Release Process

11.1 Branching and environments

- `main`: production
- `develop`: staging
- Preview environments per PR

11.2 CI pipeline

1. Lint + format check
2. Unit/integration tests
3. Security scan (dependency + secret scan)
4. Build artifact validation
5. Deploy to staging
6. Smoke tests
7. Promote to production

11.3 Render setup

- Keep auto-deploy from GitHub enabled.

- Add post-deploy smoke check hitting:
- `/api/ping`
- `/api/query-categories`
- `/` render check

12) Data Governance for Team Workflows (Master Workbook Vision)

For shared calling/enrichment workflows:

- Use server-backed canonical dataset, not shared direct-edited files.
- Support exports per project/user as generated views.
- Track status changes as events:
- contacted
- invalid
- duplicate
- preferred
- replaced

This avoids concurrent edit collisions common in shared spreadsheets.

13) Roadmap (Recommended Sequence)

Phase 1: Stabilize

- Lock API contracts and input validation
- Add structured logging + error IDs
- Add smoke tests

Phase 2: Persist

- Migrate preferred/suppressed/search memory to Postgres
- Add migration scripts from JSON files

Phase 3: Scale

- Add caching layer
- Add async enrichment worker
- Add rate limits and quotas

Phase 4: Enterprise readiness

- Multi-project scoping
- Role-based access
- Full audit trail

- Admin controls for suppression/preferred governance

14) Acceptance Criteria for “Production-Grade v1”

- No file-based mutable JSON persistence in runtime path
- Automated tests run in CI with required pass gates
- Structured logs and health checks in place
- API rate limits and validation enabled
- Key management policy documented and enforced
- One-click rollback path documented
- SLO dashboard and alerting configured